

Multi-tenancy: Multiple users sharing infrastructure safely.

- Elasticity: Ability to scale resources up or down as needed.
- Provisioning: Creating and allocating computing resources.
- Isolation: Keeping workloads separate for security and stability.

3) Key terms

- Namespace: Kernel feature that isolates system resources.
- Process namespace: Isolates process IDs.
- Network namespace: Isolates network interfaces.
- Mount namespace: Isolates filesystem views.

Slide 20/76 — Overlay filesystem (OverlayFS idea)

1) Core idea (easy paragraph)

Overlay-style filesystems allow multiple read-only layers to be combined with a writable layer, appearing as a single filesystem to the container. This approach is commonly used to implement container image layering efficiently

2) Key points

- Combines multiple layers into one view.
- Read-only image layers + writable container layer.
- Efficient storage and performance.
- Widely used for container images.
- Simplifies image distribution.

3) Key terms

- Overlay filesystem: Filesystem that merges multiple layers.
- Read-only layer: Immutable image layer.
- Copy-on-write: Writing changes without modifying original data.

Slide 22/76 — Docker network types (high-level)

1) Core idea (easy paragraph)

Docker supports different network modes to meet different needs. Some modes prioritize isolation, others simplicity or performance. Understanding these modes helps you decide how containers communicate with each other and the outside world.

2) Key points

- Bridge networks are common by default.
 - 2) Key points
 - • Virtual bridge connects containers.
 - • Containers get private IP addresses.
 - • Traffic is routed via the bridge.
 - • Port mapping enables external access.
 - • Provides isolation from host network.
 - • Default networking mode in many setups.
 -
 -
 - 3) Key terms
 - • Virtual bridge: Software-based network switch.
 - • Private IP: IP address not directly exposed externally.
 - • Port forwarding: Redirecting traffic to a container.
- Host networking removes network isolation.
 - 2) Key points
 - • Container shares host IP and ports.
 - • No network namespace isolation.
 - • Fewer networking layers → lower latency.
 - • Increased security risk.
 - • Port conflicts can occur.
 - • Typically used sparingly.
 - 3) Key terms

- • Host networking: Container uses host network directly.
- • Latency: Delay in network communication.
- • Port conflict: Two services trying to use the same port.
- None network disables networking entirely.
- Different modes trade isolation for simplicity/performance.
- Network choice affects security and accessibility.

3) Key terms

- Bridge network: Default isolated virtual network.
- Host network: Container shares host's network stack.
- None network: Container has no network access.

🔍 **Lifecycle:** تسلسل الحالات التي تمر بها الحاوية.

🔍 **Ephemeral:** قصير العمر حسب التصميم.

🔍 **Stopped container:** حاوية موجودة ولكنها لا تعمل.

🙏 تمام يا فادي

ده مش عايز حفظ، هو عايزك تمشي بخطوات تفكير واضحة وتربط بين المشكلة → القرار → السبب Design سؤال الـ هشرحلك إزاي تحل النوع ده من الأسئلة في الامتحان وبعدين أدليك إجابة نموذجية تمشي عليها.

Design؟ أولاً: إزاي تفكر في سؤال

:غالبًا يبقى فيه Design أي سؤال

1. **(Scenario) وضع/مشكلة**

2. **(Constraints) قيود**

○ Legacy app

○ Budget محدود

- Security
- Traffic

3. قرارات تصميم (Design Decisions)

4. تبرير القرار (Why?)

📌 القاعدة الذهبية:

scenario كل قرار لازم يبقى وراه سبب منطقي مرتبط بالـ

1 VM vs Container Architecture

إزاي تجاوب؟

الخطوة 1: افهم السيناريو

- Application قديم (Legacy)
- Refactoring budget محدود
- مش عايزين نغير الكود كتير

الخطوة 2: القرار

👉 Virtual Machines نستخدم

الخطوة 3: التبرير

- كامل OS بتشغل VM الـ
 - التطبيق يشتغل زي ما هو
 - مفيش تعديل كبير على الكود
-

إجابة نموذجية

Given the legacy nature of the application and the limited refactoring budget, virtual machines are the most suitable choice. VMs allow the application to run on a full operating system environment with minimal changes, while still benefiting from cloud infrastructure such as scalability and availability. Containers can be considered later if refactoring becomes feasible.

Diagram مهم في (design بسيط)

Application

↓

Guest OS

↓

Virtual Machine

↓

Hypervisor

↓

Cloud Infrastructure

2 Kubernetes Deployment

إزاي تجاوب؟

الخطوة 1: المطلوب

- API
- عالي Traffic
- Scalability
- Cost control

الخطوة 2: المكونات

- Deployment
 - Replicas
 - Service
 - Autoscaling
-

إجابة نموذجية

The application should be deployed using a Kubernetes Deployment with multiple replicas to ensure high availability. A Kubernetes Service will expose the API and distribute incoming traffic across the replicas. Horizontal Pod Autoscaling can be enabled to automatically scale the number of replicas based on traffic demand, ensuring efficient resource usage and cost control.

Diagram بسيط

Client



Service (Load Balancer)



Pod 1 Pod 2 Pod 3

(Replica Pods)

3 Migration Strategy

إزاي تجاوب؟

الخطوة 1: نوع التطبيق

- Legacy
- قليلة Cloud experience

Strategy الخطوة 2: أفضل

👉 Lift and Shift Migration

ليه؟

- أسرع
- أقل مخاطرة
- مش محتاج خبرة كبيرة
- بعدين Optimization

إجابة نموذجية

A lift-and-shift migration strategy is appropriate in this case. It minimizes risk and requires minimal cloud expertise, allowing the application to be moved quickly to the cloud. Performance and cost optimizations can be addressed in later phases once the application is stable in the cloud environment.

Diagram

On-Prem Server



Cloud VM

(Same App, Same OS)

4 Security Responsibility (Shared Responsibility Model)

إزاي تجاوب؟

الخطوة 1: افصل المسؤوليات

- Provider
 - Organization
-

مسؤول عن Provider:

- Physical servers
- Networking
- Hypervisor
- Data center security

مسؤولة عن Organization:

- Data
- IAM

- Access control
- App security
- Encryption

إجابة نموذجية

Security follows a shared responsibility model. The cloud provider is responsible for securing the underlying infrastructure and platform, including physical servers, networking, and virtualization. The organization remains responsible for application-level security, access control, identity management, and data protection. Strong IAM policies and encryption should be implemented to protect customer data and meet compliance requirements.

Diagram

Customer Responsibilities:

- Application
- Data
- IAM
- Encryption

Cloud Provider:

- Hardware
- Network
- Hypervisor
- Data Center

 **Template** شكل الإجابة في الامتحان

Design Question: لو جالك

1. Architecture choice

→ VM / Container + السبب

2. Deployment model

→ Kubernetes (Deployment + Service + Autoscaling)

3. Migration approach

→ Lift & Shift + Why

4. Security model

→ Shared Responsibility

👉 نصيحة مهمة

%حتى لو مش فاهم 100:

- واضح decision اكتب
- scenario اربطه بـ
- بسيط diagram ارسم

👉 ده لوحده بييجيب درجات عالية جدًا

لو حابب:

- كامل وتحله لوحدهك Design أدبك سؤال
- قصيرة للحفظ Model Answer أو أعمالك
- بسرعة في الامتحان diagram أو أشرحك إزاي ترسم

👉 قولّي نكمل بإيه

Here's a **quick, exam-friendly revision of Kubernetes (K8s)** 👉

Short, clear, and focused on **concepts + components + storage + services**.

Kubernetes (K8s) — Quick Revision

1 What is Kubernetes?

Kubernetes is a container orchestration platform used to:

- Deploy applications
- Scale them up/down
- Manage failures
- Automate container operations

👉 It mainly manages **containers (usually Docker)**.

2 Core Kubernetes Concepts

Cluster

- A Kubernetes cluster = **Control Plane + Worker Nodes**
-

Node

- A machine (VM or physical)
- Runs application workloads

Two types:

- **Control Plane Node**
 - **Worker Node**
-

Pod

- Smallest unit in Kubernetes
- Contains:
 - One or more containers
 - Shared network & storage

📌 Pods are **ephemeral** (can die and be recreated).

3 Control Plane Components

API Server

- Entry point to the cluster
- All commands go through it

Scheduler

- Decides **which node** a Pod runs on

Controller Manager

- Ensures desired state is met
- Example: if a Pod dies → create a new one

etcd

- Key-value database
- Stores cluster state & configuration

4 Worker Node Components

kubelet

- Agent on each node
- Ensures Pods are running

Container Runtime

- Runs containers
- Example: Docker, containerd

kube-proxy

- Handles networking
- Enables Services & load balancing

5 Workload Objects

Deployment

- Manages Pods declaratively
- Supports:
 - Replicas
 - Rolling updates
 - Self-healing

 Most common object

Deployment

↓

ReplicaSet

↓

Pods

ReplicaSet

- Ensures a fixed number of Pods are running

StatefulSet

- For stateful apps (DBs)
- Stable:
 - Network identity
 - Storage

DaemonSet

- Runs one Pod on **every node**
- Example:
 - Logging

- Monitoring agents
-

6 Services (Networking)

Why Services?

- Pods change IPs
 - Services provide **stable access**
-

Types of Services

ClusterIP (default)

- Internal access only
- Pod-to-Pod communication

NodePort

- Exposes app on node IP + port
- Accessible externally

LoadBalancer

- Uses cloud load balancer
- Best for production

ExternalName

- Maps service to external DNS
-

Traffic Flow Example

Client

↓

Service

↓

Pods (Load Balanced)

7 Storage in Kubernetes

Volume

- Storage attached to a Pod
- Lives as long as the Pod

 Deleted when Pod is deleted

Persistent Volume (PV)

- Actual storage resource
- Created by:
 - Admin
 - Cloud provider

Examples:

- AWS EBS
- NFS
- Azure Disk

Persistent Volume Claim (PVC)

- Request for storage by a Pod
- Defines:
 - Size
 - Access mode

 Pod uses **PVC**, not PV directly

PV-PVC Relationship

Pod

↓

PVC (request)

↓

PV (real storage)

Access Modes

- **ReadWriteOnce (RWO)** – one node
 - **ReadOnlyMany (ROX)** – many nodes (read)
 - **ReadWriteMany (RWX)** – many nodes (read/write)
-

8 Config & Secrets

ConfigMap

- Non-sensitive config data
- Example:
 - URLs
 - App configs

Secret

- Sensitive data
 - Example:
 - Passwords
 - Tokens
 - API keys
-

9 Scaling & Availability

Horizontal Pod Autoscaler (HPA)

- Scales Pods based on:

- CPU
- Memory
- Metrics

Self-Healing

- If Pod crashes → recreated
- If Node fails → Pods rescheduled

Kubernetes Summary (Very Important)

Concept	Purpose
Pod	Smallest unit
Deployment	Manages Pods
Service	Stable networking
PV	Actual storage
PVC	Storage request
Volume	Pod-level storage
Node	Runs Pods
Cluster	Entire K8s system

Exam Tip (Gold)

If asked “**Design on Kubernetes**”, always mention:

- Deployment
- Multiple replicas
- Service for exposure
- Autoscaling

- PVC for persistent data
-

If you want next:

- **VM vs K8s vs Docker (comparison table)**
- **K8s diagrams for exam**
- **K8s MCQ revision questions**

Just tell me 👍